

Justin Andre

COIN 496 Senior Portfolio

12/07/2010

Program Introduction Page

Program Name: Petshop
Creation Date: Fall 2010
Last Modified Date: Fall 2010
COIN Course: COIN 326
Grade Received: B
Comments: open with netbeans if possible

Program Name: Indexed Multi List (using Linked Lists)
Creation Date: Spring 2010
Last Modified Date: Spring 2010
COIN Course: COIN 315
Grade Received: B
Comments: Run using Visual Studios (unzip if necessary first)

Program Name: Justin's Martial Arts Store
Creation Date: Fall 2009
Last Modified Date: Fall 2009
COIN Course: COIN 332
Grade Received: B
Comments: Run using visual studios (unzip if necessary first)

```
#include "IndexedMultiList.h"
#include <iostream>
using namespace std;
/*
    Student Name: Justin Andre
    Program Name: Indexed Multi List (using linked list)
    Creation Date: Spring 2010
    Last Modified Date: Spring 2010
    COIN Course: 315
    Grade Received: B
    Comments regarding design:
*/
IndexedMultiList::IndexedMultiList(){
    head = 0;
}

IndexedMultiList::~IndexedMultiList(){
    Node * trail = head;
    if(!head){
        return;           // check for empty list
    }
    head = head ->next;
    while(head){
        delete trail;
        trail = head;
        head = head->next;
    }
    delete trail;
}

bool IndexedMultiList::empty(){
    Node * cur = head;
    if(head == NULL){
        return true;
    }
    else {
        return false;
    }
}

int IndexedMultiList::uniqueSize(){
    Node * cur = head;
    int i = 0;
    while(cur != NULL){
        cur = cur ->next;
        i++;
    }
    return i;
}

int IndexedMultiList::size(){
    Node * cur = head;
    int i = 0;
    while(cur != NULL){
        cur = cur ->next;
        i = i + cur->data;
    }
    return i;
}

int IndexedMultiList::find(char val){
    Node * cur = head;
    int i = 0;
    while(cur != NULL){
        cur = cur->next;
        i++;
        if(cur->letter == val){
            return i;
        }
    }
}
```

```
    if (cur == NULL){
        return -1;
    }
}

int IndexedMultiList::countItem (char val) {
    int i = 0;
    Node * cur = head;
    while (cur != NULL){
        cur = cur->next;
        if (cur->letter == val){
            i++;
        }
    }
    return i;
}

int IndexedMultiList::totalItemCount (char val){
    int i = 0;
    Node * cur = head;
    while (cur != 0){
        cur = cur->next;
        if (cur->letter == val){
            i = i + cur->data;
        }
    }
    return i;
}

int IndexedMultiList::countPosition(int index){
    int i = 0;
    Node * cur = head;
    {
        while (cur != 0 && i < index){
            cur = cur->next;
            i++;
        }
    }
    return i;
}

bool IndexedMultiList::getAt(int index, char &out){
    int i = 0;
    Node * cur = head;
    while (cur != 0 && i < index){
        cur = cur->next;
        i++;
    }
    if (cur != 0){
        out = cur->letter;
        return true;
    }
    return false;
}

void IndexedMultiList::print(){
    Node * cur = head, * trail = NULL;
    for (Node * cur=head; cur != 0; cur=cur->next) {
        cout <<cur->letter<<" "<<cur->data<<endl;
    }
}

bool IndexedMultiList::insert(int index, char val, int count){
    Node * cur = head, * trail = NULL;
    int i = 0;
    if (index < 0 || count < 1){ // validate index and count
        return false;
    }
    if (head == NULL){ // check for empty list and if insert intends to add to beginning
        head = new Node (val, count, head);
        return (head != NULL);
    }
}
```

```
while(cur != NULL && i < index){ // step through list insert into middle or end based on index
    trail = cur;
    cur = cur->next;
    i++;
}
if(trail==0){
    head = new Node(val, count, head);
    if(cur == NULL){ // check to see if went off the end of the list
        return false;
    }
    head = cur;
    return true;
}
if(i == index){ // insert only if index is found
    Node * newData = new Node(val, count);
    newData->next = cur;
    trail->next = newData;
    if(newData == NULL){
        return false;
    }
    return true;
}
}

bool IndexedMultiList::change(int index, int amount){
    int i = 0;
    Node * cur = head, * trail = 0;
    if(!empty()){
        while(cur != 0 && i < index){
            trail = cur;
            cur = cur->next;
            i++;
        }
        if(cur == 0){
            return false;
        }
        if(i == index){
            cur->data = cur->data + amount;
            if(cur->data <= 0){
                trail->next = cur->next;
                delete cur;
            }
            return true;
        }
    }
}

bool IndexedMultiList::update(char val, int amount){
    Node * cur = head, * trail = 0;
    while(cur != NULL && cur->letter != val){
        trail = cur;
        cur = cur->next;
    }
    if(cur == 0){
        return false;
    }
    if(cur->letter == val){
        cur->data = cur->data + amount;
        if(cur->data <= 0){
            trail->next = cur->next;
            delete cur;
        }
        return true;
    }
}

bool IndexedMultiList::remove(char val){
    Node * cur = head, * trail = 0;
    if(!empty()){
```

```
    while(cur!= NULL && cur->letter != val){
        trail = cur;
        cur = cur->next;
    }
    if(cur == 0){
        return false;
    }
    if(cur->letter == val){
        cur->data = cur->data -1;
        if(cur->data <=0){
            trail->next = cur->next;
            delete cur;
        }
        return true;
    }
}

int IndexedMultiList::removeIndex(int index){
    int i = 0;
    Node * cur = head, * trail = 0;
    if(!empty()){
        while(cur != 0 && i < index){
            trail = cur;
            cur = cur->next;
            i++;
        }
        cur->data = cur->data -1;
        return cur->data;
    }
}

int IndexedMultiList::removeAll(char val){
    int i = 0;
    Node * cur = head, * trail = 0;
    if(!empty()){
        while(cur!= NULL && cur->letter != val){
            trail = cur;
            cur = cur->next;
        }
        if(cur->letter == val){
            i = cur->data;
            trail->next = cur->next;
            delete cur;
        }
    }
    return i;
}
```

```

#include <iostream>
using namespace std;

#ifndef INDEXEDMULTILIST_H
#define INDEXEDMULTILIST_H
/*
    Student Name: Justin Andre
    Program Name: Indexed Multi List (using linked list)
    Creation Date: Spring 2010
    Last Modified Date: Spring 2010
    COIN Course: 315
    Grade Received: B
    Comments regarding design:
*/
class IndexedMultiList {
public:
    //Constructors/destructors
    IndexedMultiList();//creates an empty list capable of storing

    ~IndexedMultiList();    //destructor ONLY if needed

    //accessors
    bool empty();//returns true iff (if and only if) list is empty
    int uniqueSize();//returns number of positions in the list in use
    int size();//returns total number of items in list using counts
    //If 2 A, 3 C, 1 D are inserted into an IndexedMultiList iml then
    //impl.uniqueSize() would return 3 and impl.size() would return 6.
    int find(char val);
    //returns index of first position with val in it or -1 if not found
    // index is NOT a spot in an array
    int countItem (char val);
    //returns number of locations val is in the list (vacuously 0 if not //found)
    int totalItemCount (char val);
    /*returns number of val in the list (vacuously 0 if not found)
    Example: there are 3 A's in the first position, 6 B's in the second and 4 A's in the third
    position.
    find('A') returns 0, find('B') returns 1 and find('C') returns -1
    countItem('A') returns 2 (it is in the first and third location)
    countItem('B') returns 1, countItem('C') returns 0
    totalItemCount('A') returns 7, totalItemCount('B') returns 6,
    totalItemCount('C') returns 0
    */

    int countPosition(int index);//returns number of items at position index (0 based) or
    0 if index is invalid
    bool getAt(int index, char &out);
    /*if position index is in use, out is set to char at that position and true is returned,
    otherwise false is returned. This is kind of like [] for an array but with error
    checking*/

    void print();
    //prints out the content of the list in order by position, see below

    //mutators
    bool insert(int index, char val, int count);
    /*insert count number of val into the list at position index
    returns true iff val was able to fit in list and index was valid and count >=1
    valid indices range from 0 up to unique size (inclusive). So for an empty list, 0 is the
    only valid index.
    insert should do nothing if index is invalid
    If the list has 3 A's in the first position, then 0 and 1 are the only valid indices.
    return false only if out of memory, not if using up 20 items
    */
    bool change(int index, int amount);
    /*add amount to item in position index
    returns true iff index was valid

```

```
change should do nothing if index is invalid
amount could be negative, if adding amount to the current count results in a count of <= 0
    then position should be removed
For example, if there are 3 A's at index 0 and 2 B's at index 1 in the list iml then
    iml.change(0, -4)
    would remove all the A's and the 2 B's would be at index 0.
*/

    bool update(char val, int amount); //finds val in list and adds amount (which could be
        negative)
//return true iff found, works similarly to change but is based on the
//character to find instead of the position. If val is in multiple //locations within the
    list, then only change the first location

    bool remove(char val); //removes 1 instance of val from list, //returns true iff val
        was in list, if val is in multiple locations,
//then only remove from the first location

    int removeIndex(int index); //remove item in position index, returns count //of items
        removed or -1 if index not in use

    int removeAll(char val); //removes all instances of val from list
//returns number of items removed, which could be 0, if val is in //multiple locations it
    should remove only the first location

private:
//put whatever methods or data you need in here,
//you must put the 1-dimensional array here
struct Node {
    Node(char l, int d, Node *n=0) {letter=l; data=d; next=n;}
    int data;
    char letter;
    Node *next;
};
Node * head;

};

#endif
```



```
#include "IndexedMultiList.h"
#include <iostream>
using namespace std;
/*
    Student Name: Justin Andre
    Program Name: Indexed Multi List (using linked list)
    Creation Date: Spring 2010
    Last Modified Date: Spring 2010
    COIN Course: 315
    Grade Received: B
    Comments regarding design:
*/
void main(){

    IndexedMultiList list1;
    list1.insert(5, 'a', 9);
    list1.insert(1, 'b', 4);
    list1.insert(2, 'c', 3);
    list1.removeAll('c');
    list1.print();

}
```

```
/*
    Student Name: Justin Andre
    Program Name: PetShop
Creation Date: Fall 2010
Last Modified Date: Fall 2010
COIN Course: COIN 326
Grade Received: B
Comments regarding design:
*/
package petshop;
/**
 * Fish inherits from pet like mammal does, but it needs only one more
 * variable.
 * @author Justin Andre
 */
public class Fish extends Pet{
    String last_tank_change;
    public Fish(String petname, float startcost, float curcost, String found,
        String adopted, String Tchanged){
        super(petname, startcost, curcost, found, adopted);

        Tchanged = last_tank_change;
    }
    /** @param tank change is to keep track of the water for the handlers to
     * help keep the fish healthy. */
    public void setlast_tank_change(String Tchanged){
        last_tank_change = Tchanged;
    }
    /** @return each of the variable names after setting the input from the
     * user to match the data variables. */
    public String getDateIn(){
        return last_tank_change;
    }
    /** @return each of the variable names to be printed to the screen
     * as output. */
    public String toString(){
        return String.format("Pet ", Name, "Starting Price: ", StartingPrice,
            "Current Price ", CurrentPrice, "Date In ", DateIn, "Date Adopted ",
            DateAdopted, "Last Tank Change ", last_tank_change);
    }
}
```

```
/*
    Student Name: Justin Andre
    Program Name: PetShop
Creation Date: Fall 2010
Last Modified Date: Fall 2010
COIN Course: COIN 326
Grade Received: B
Comments regarding design:

*/
package petshop;
/**
 * A pet shop that receives fish and mammals and allows customers to
 * adopt them.
 * @author Justin Andre
 * @version 6.9.1
 */
public class Main {
    /**
     * @param args the command line arguments
     */
    public static void main(String[] args) {

        // TODO code application logic here
    }
}
```

```
/*
    Student Name: Justin Andre
    Program Name: PetShop
Creation Date: Fall 2010
Last Modified Date: Fall 2010
COIN Course: COIN 326
Grade Received: B
Comments regarding design:
*/
package petshop;
/**
 * An class that inherits from pet but has some unique variables that fish
 * does not.
 * @author Justin Andre
 */
public class Mammal extends Pet{
    protected int age;
    protected int weight;
    protected int height;
    protected String last_vet_visit;
    /** @param age is more relevant to mammals than fish in this case,
     * therefore it is included for mammals but not fish.
     * @param weight is more relevant to mammals than fish in this case,
     * therefore it is included for mammals but not fish.
     * @param height is more relevant to mammals than fish in this case,
     * therefore it is included for mammals but not fish.
     * @param vet visit is important to keep shot records up to date and
     * to track medical history for future adoption.
     */
    public Mammal(String petname, float startcost, float curcost,
        String found, String adopted, int mammal_age, int mammal_weight,
        int mammal_height, String vet_visit){
        super(petname, startcost, curcost, found, adopted);
        age = mammal_age;
        weight = mammal_weight;
        height = mammal_height;
        last_vet_visit = vet_visit;
    }
    /** @return each of the variable names after setting the input
     * from the user to match the data variables.
     */
}
```

```
public void settage(int mammal_age){
    age = mammal_age;
}
public int getage(){
    return age;
}
public void setweight(int mammal_weight){
    weight = mammal_weight;
}
public int getweight(){
    return weight;
}
public void setheight(int mammal_height){
    height = mammal_height;
}
public int getheight(){
    return height;
}
public void setvisit(String vet_visit){
    last_vet_visit = vet_visit;
}
public String setvisit(){
    return last_vet_visit;
}
/** @return each of the variable names to be printed to the
    * screen as output. */
public String toString(){
    return String.format("Pet ", Name, "Starting Price: ", StartingPrice,
        "Current Price ", CurrentPrice, "Date In ", DateIn, "Date Adopted ",
        DateAdopted, "Age ", age, "Weight ", weight, "Height ", height,
        "Last vet visit ", last_vet_visit);
}
}
```

```
/*
    Student Name: Justin Andre
    Program Name: PetShop
Creation Date: Fall 2010
Last Modified Date: Fall 2010
COIN Course: COIN 326
Grade Received: B
Comments regarding design:
    */
/**
 * Creates an instance of a pet to hold its basic information.
 * @author Justin Andre
 */
package petshop;
/**
 *
 * @author Justin
 */
public class Pet {
protected String Name;
protected float StartingPrice;
protected float CurrentPrice;
protected String DateIn;
protected String DateAdopted;
    /** @param petname is the name of the pet.
 * @param startcost will be the starting price of the pet when the shop
 * receives it.
 * @param curcost will be the current cost of the pet if different from the
 * starting cost.
 * @param found is the date the shop received the animal for information
 * purposes.
 * @param adopted is the date that the shop found a home for the pet.
 */
    public Pet(String petname, float startcost, float curcost, String found,
        String adopted){
        Name = petname;
        StartingPrice = startcost;
        CurrentPrice = curcost;
        DateIn = found;
        DateAdopted = adopted;
    }
}
```

```
/** @return each of the variable names after setting the input from the
 * user to match the data variables.*/
public void setName(String petname){
    Name = petname;
}
public String getName(){
    return Name;
}
public void setStartPrice(float startcost){
    StartingPrice = startcost;
}
public float getStartPrice(){
    return StartingPrice;
}
public void setCurPrice(float curcost){
    CurrentPrice = curcost;
}
public float getCurPrice(){
    return CurrentPrice;
}
public void setDateIn(String found){
    DateIn = found;
}
public String getDateIn(){
    return DateIn;
}
public void setadopted(String adopted){
    DateAdopted = adopted;
}
public String getadopted(){
    return DateAdopted;
}
}
```

```
Imports System
Imports System.Data
Imports System.Data.Sql
Imports System.Data.SqlClient
Partial Class Checkout
    Inherits System.Web.UI.Page
```

```
Protected Sub Button1_Click(ByVal sender As Object, ByVal e As System.EventArgs) Handles Button1.Click
    FormsAuthentication.SignOut()
```

```
    Response.Redirect("Userlogin.aspx")
```

```
End Sub
```

```
End Class
```



```
<%@ Page Language="VB" AutoEventWireup="false" CodeFile="Checkout.aspx.vb" Inherits="Checkout" %>
```

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
```

```
<html xmlns="http://www.w3.org/1999/xhtml">
```

```
<head runat="server">
```

```
  <title>Untitled Page</title>
```

```
</head>
```

```
<body>
```

```
  <form id="form1" runat="server">
```

```
    <div>
```

```
      This is the checkout page<br />
```

```
      <br />
```

```
      <br />
```

```
      <asp:Label ID="Label1" runat="server" Font-Size="Larger" Text="User Invoice"></asp:Label>
```

```
      <br />
```

```
      <br />
```

```
      <br />
```

```
      <br />
```

```
      <asp:Button ID="Button1" runat="server" Text="Back to Login" />
```

```
    </div>
```

```
  </form>
```

```
</body>
```

```
</html>
```

```
Imports System
Imports System.Data
Imports System.Data.Sql
Imports System.Data.SqlClient
Partial Class LoginPage
    Inherits System.Web.UI.Page

    Protected Sub Button1_Click(ByVal sender As Object, ByVal e As System.EventArgs) Handles Button1.Click
        Response.Redirect("Userlogin.aspx")
    End Sub
End Class
```

```
<%@ Page Language="VB" AutoEventWireup="false" CodeFile="CreateAccount.aspx.vb" Inherits="LoginPage" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-
transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
    <title>Untitled Page</title>
</head>
<body>
    <form id="form1" runat="server">
        <div>

            <asp:CreateUserWizard ID="CreateUserWizard1" runat="server" BackColor="#FFFBD6"
                BorderColor="#FFDFAD" BorderStyle="Solid" BorderWidth="1px"
                CancelDestinationPageUrl="~/Userlogin.aspx"
                ContinueDestinationPageUrl="~/Userlogin.aspx" Font-Names="Verdana"
                Font-Size="0.8em">
                <SideBarStyle BackColor="#990000" Font-Size="0.9em" VerticalAlign="Top" />
                <SideBarButtonStyle ForeColor="White" />
                <ContinueButtonStyle BackColor="White" BorderColor="#CC9966"
                    BorderStyle="Solid" BorderWidth="1px" Font-Names="Verdana"
                    ForeColor="#990000" />
                <NavigationButtonStyle BackColor="White" BorderColor="#CC9966"
                    BorderStyle="Solid" BorderWidth="1px" Font-Names="Verdana"
                    ForeColor="#990000" />
                <HeaderStyle BackColor="#FFCC66" BorderColor="#FFFBD6" BorderStyle="Solid"
                    BorderWidth="2px" Font-Bold="True" Font-Size="0.9em" ForeColor="#333333"
                    HorizontalAlign="Center" />
                <CreateUserButtonStyle BackColor="White" BorderColor="#CC9966"
                    BorderStyle="Solid" BorderWidth="1px" Font-Names="Verdana"
                    ForeColor="#990000" />
                <HeaderTextStyle BackColor="#990000" Font-Bold="True" ForeColor="White" />
                <WizardSteps>
                    <asp:CreateUserWizardStep runat="server" />
                    <asp:CompleteWizardStep runat="server" />
                </WizardSteps>
            </asp:CreateUserWizard>
            <br />
            <asp:Button ID="Button1" runat="server" Text="Back to User Log In" />

        </div>
    </form>
</body>
</html>
```

```
Imports System
```

```
Imports System.Data
```

```
Imports System.Data.Sql
```

```
Imports System.Data.SqlClient
```

```
Partial Class Purchase
```

```
    Inherits System.Web.UI.Page
```

```
    Protected Sub Button1_Click(ByVal sender As Object, ByVal e As System.EventArgs) Handles Button1.Click
        FormsAuthentication.SignOut()
        Response.Redirect("Userlogin.aspx")
    End Sub
```

```
    Protected Sub Button2_Click(ByVal sender As Object, ByVal e As System.EventArgs) Handles Button2.Click
        Dim reader As SqlDataReader
```

```
    End Sub
```

```
    Protected Sub Button3_Click(ByVal sender As Object, ByVal e As System.EventArgs) Handles Button3.Click
        Dim counter, price As Integer
```

```
        If (counter = 0) Then
            Label1.Text = DropDownList1.SelectedItem.ToString
            counter = counter + 1
            price = price + DropDownList1.SelectedValue
            Label5.Text = "$ " & price
        ElseIf (counter = 1) Then
            Label2.Text = DropDownList1.SelectedItem.ToString
            counter = counter + 1
            price = price + DropDownList1.SelectedValue
            Label5.Text = "$ " & price
```

```
        ElseIf (counter = 2) Then
            Label3.Text = DropDownList1.SelectedItem.ToString
            counter = counter + 1
            price = price + DropDownList1.SelectedValue
            Label5.Text = "$ " & price
```

```
        ElseIf (counter = 3) Then
            Label3.Text = DropDownList1.SelectedItem.ToString
            counter = counter + 1
            price = price + DropDownList1.SelectedValue
            Label5.Text = "$ " & price
```

```
        ElseIf (counter = 4) Then
            Label4.Text = DropDownList1.SelectedItem.ToString
            counter = counter + 1
            price = price + DropDownList1.SelectedValue
            Label5.Text = "$ " & price
```

```
        ElseIf (counter = 5) Then
            DropDownList1.Enabled = False
            Button3.Enabled = False
            Label5.Text = "$ " & price
```

```
    End If
```

```
End Sub
```

```
End Class
```

➤

```
</body>  
</html>
```

```
Imports System
Imports System.Data
Imports System.Data.Sql
Imports System.Data.SqlClient
Partial Class _Default
    Inherits System.Web.UI.Page

    Protected Sub Button1_Click(ByVal sender As Object, ByVal e As System.EventArgs) Handles Button1.Click
        Response.Redirect("CreateAccount.aspx")
    End Sub

End Class
```

```
<%@ Page Language="VB" AutoEventWireup="false" CodeFile="Userlogin.aspx.vb" Inherits="_Default" %>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-
transitional.dtd">

<html xmlns="http://www.w3.org/1999/xhtml">
<head runat="server">
  <title>Untitled Page</title>
</head>
<body>
  <form id="form1" runat="server">
    <div>

      Welcome to Justin's Martial Arts Supply Store!<br />
      Please log in to continue<br />
      <asp:Login ID="Login1" runat="server" BackColor="#FFFBD6" BorderColor="#FFDFAD"
        BorderPadding="4" BorderStyle="Solid" BorderWidth="1px"
        DestinationPageUrl="~/Purchase.aspx" Font-Names="Verdana" Font-Size="0.8em"
        ForeColor="#333333" TextLayout="TextOnTop">
        <TextBoxStyle Font-Size="0.8em" />
        <LoginButtonStyle BackColor="White" BorderColor="#CC9966" BorderStyle="Solid"
          BorderWidth="1px" Font-Names="Verdana" Font-Size="0.8em" ForeColor="#990000" />
        <InstructionTextStyle Font-Italic="True" ForeColor="Black" />
        <TitleTextStyle BackColor="#990000" Font-Bold="True" Font-Size="0.9em"
          ForeColor="White" />
      </asp:Login>
      <br />
      <br />
      Dont have an account? Click
      <asp:Button ID="Button1" runat="server" Text="Here!" />

    </div>
  </form>
</body>
</html>
```


Ethics Paper Introductory Page

Coin 334 Ethics Paper
Grade: A

Coin 332 Ethics Paper
Grade: C

Coin 315 Ethics Paper
Grade: C

Justin Andre

7/30/2010

Coins 334

Ethics Paper Assignment

Responsibility

The Rutan model 61 Long-EZ is an unusual home built aircraft by Burt Rutan's Rutan Air Factory. The aircraft is designed for long-range fuel efficient trips. It can fly for over 10 hours, covering 1600 miles using 52 gallons of fuel. If modified properly, this aircraft can cover distances of up to 4,800 miles without refueling. The pilot sits in a semi-reclined seat and controls the Long-EZ by means of a side-stick controller situated on the right-hand console. In addition to having an airbrake on the underside, the twin tail's wing-tip rudders can be deflected outwards to act as auxiliary airbrakes. The aircraft will not stall in the manner of a conventional aircraft since, if the Long-EZ reaches too low a speed, the front wing will stall and lower the aircraft nose until speed is regained (Wikipedia Rutan-Long EZ p 4).

At the time this plane was an experimental kit aircraft. A kit aircraft can be modified as the builder sees fit. It was designed, however, by Burt Rutan, the aerospace engineer famous for designing the Voyager, the world's first aircraft to circumnavigate the globe without refueling (When Interfaces Kill- p 2). On October 12, 1997 John Denver, a popular American singer and songwriter, died when his Long-EZ aircraft crashed into the Pacific Ocean off the coast of California. The cause of the accident can be purely credited to a clumsily designed user interface inside the cockpit.

When designing an aircraft, safety is the most important aspect to consider. This is not the case for automobiles, which had to be government regulated to include safety features such as windshield wipers, headlights, and safety belts. A fuel leak for an automobile is not as severe an issue as it is for an airplane. The difference is the operator usually has time to pull over and get out of the car. In an aircraft the time it takes to land and get out the leak becomes a catastrophic disaster for everyone on board. As a result,

cockpits contain fuel shutoff valves so that fire and smoke do not impede the pilot's ability to land the plane as soon as possible. Additionally, most aircraft contain options to shut off the left fuel tank, the right fuel tank, or both (When Interfaces Kill- p 5-7).

The Long-EZ is known for its terrible interface. The fuel shutoff valve has three options: off, left, and right (not both). Burt Rutan's designs specify that the valve is to be placed on the front panel of the aircraft, making it easy to switch among the options. The builder of the aircraft, for a specific reason, decided to place the valve behind the pilot's left shoulder. By placing the valve behind the pilot's compartment, on the other side of the back firewall, with only a long rod leading to the handle behind the pilot's left shoulder, he avoided running the gas lines through the passenger compartment, eliminating any possibility of a gas leak occurring inside the compartment (When Interfaces Kill- p 8-9). This was a terrible idea in terms of user interface design, because the only way to switch tanks was to let go of the controls, twist your head to the left to look behind you, reach over your left shoulder with your right hand, find the valve, and turn it. As the National Transportation Safety Board (NTSB) discovered, it was difficult to do this without bracing yourself with your right foot—which in turn presses the right rudder pedal all the way to the floor. And that's what killed John Denver. His plane was seen veering to the right and plunging into the ocean from only a few hundred feet up, consistent with the NTSB's reconstruction (When Interfaces Kill- p 8-9). As a side note, the builder not only placed the valve in a terrible place, it was also backwards; Turning the valve to the right would shut off the left fuel tank lines. This ensured that anyone unfamiliar with this error would accidentally shut off the wrong fuel tank lines, and cause even more problems for themselves.

There were other interface issues as well with this model plane. Because of the way the aircraft rests on the ground, the fuel gauge tends to mislead the pilot into thinking they have more fuel than is present. In this case, the kit builder also put the fuel gauge behind the pilot, with a mirror for the pilot to observe. It should be noted that these fuel gauges are not linear and not very well labeled. This would make it near impossible to read the fuel gauge backwards, which is already misleading the pilot with a generous fuel level (When Interfaces Kill p 10-11).

So who is responsible for the crash? Fault can be traced to three sources: the designer of the aircraft, the builder of the kit, and the pilot himself. Burt Rutan's design was respectable, but anyone who designs things with options must know that someone out there will put things in unintended places (with or without recommendation). The easiest thing to point out is that if the option does not exist, the kit builder cannot put things where they should not go. It does not take an expert in building aircraft to come up with a way so that the fuel shut off valves and gauges MUST be placed up front where the pilot can readily access them without turning around. Furthermore, the valve should not require an incredible amount of strength to turn.

It is easy to point blame at the kit builder. This was the person who had the bright ideas of making a life or death situation an instant death situation. If something is recommended to be constructed a certain way, and safety is the most important factor in what is being constructed, then the builder is obligated to follow recommendations. There is no valid argument against this, but there will always be someone who, given the option, will try to do things their way. But if it is such an obvious mistake, why have the

option to make such bad decisions in the first place? Can the design simply require these two things be placed up front somehow? Surely there is a way to do this.

Finally, the pilot himself is also responsible for this tragedy. It is noted that John Denver was an experienced pilot (with over 2,700 hours of experience), so he must have known how backwards the current configurations looked (Wikipedia.com John Denver p 6). Furthermore, an experienced pilot must know the dangers involved with flying, and that piloting something that operated differently from what you are used to (based on the fact that things are visibly different from the standard cockpit) can be very dangerous. Even if regulations permitted the use of this particular aircraft, common sense would seem to take over in terms of knowing that the fuel gauge is backwards and the fuel shut off valves are not where they are supposed to be (and as a result are backwards too).

Works Cited

1. <http://www.asktog.com/columns/027InterfacesThatKill.html>
2. <http://en.wikipedia.org/wiki/Long-Ez>
3. http://en.wikipedia.org/wiki/John_Denver#Death

Justin Andre

11/20/09

Coins 332

Ethics Paper

Online Piracy of Movies

The illegal downloading of movies, music, and other data files is an unintended consequence of file sharing technology. Organizations such as the MPAA combat server hosts who promote such actions, but there is no possible way to completely remove all of the transgressors. Though these are considered crimes individually, the level of punishment should be considered minimal to moderate. Downloading movies and music is such a common thing for today's youth that it really should be thought of as a more common form of shoplifting. A movie that costs \$20 in a store can be illegally downloaded, and that is a crime. The punishment for such a crime should not exceed a small fine or temporary ban from the use of their ISP, much the same a person caught shoplifting would be banned from the store (permanently or temporarily, depending on the severity).

One could also argue that illegally downloading content is okay, but the user is at risk for acquiring viruses, spyware, or other such cyber threats. This perspective is in favor of piracy, and is probably aware of the negative consequences (and thus more prepared to not get caught or get spyware). This is an unethical approach to the subject, and ignores the simple fact that there are laws against such actions and consequences for breaking such laws. The up-side of this perspective is that it challenges those to take better security measures in preventing such piracy, which opens the market for jobs and

allows the possibility of new technologies and techniques for battling piracy to come about.

Personally, illegally downloading music and movies can be very beneficial for the artist or movie in question. This encourages consumers to buy more products associated with the producer. I know of people who have downloaded a movie illegally before its release, enjoyed it, and went to the theater to watch it again. I have read about others who download a few songs of an album to see if they like the music and then choose whether or not to buy the CD. This is an unintended consequence of file sharing technology. Are there more people who use this for their own benefit and do not later support their favorite artist/movie maker? Yes. Are musicians and actors overpaid? Yes. Do they still have loyal fans who buy T-shirts, CDs, games, movies, music, and other associated products? Yes. Should a lawsuit be brought against transgressors that simply awards money to the already rich? No.

I believe that once a product has been bought such as a movie or CD, it should be able to be downloaded by that person anytime anywhere. That is the nature of today's technology. The security measures simply need to catch up so that people cannot "shoplift" from the internet. Artists and movie makers alike can use this for their benefit. Appealing to more fans by allowing some songs or clips or related material to be free on the internet is a potentially smart move.

In short, internet piracy (in most cases) is not a big deal, though it is illegal and therefore should not be done. It is very interesting to see other people's perspectives on the matter and how one justifies their beliefs. Morally it is a healthy thing because it challenges the way we think and promotes constant re-evaluation of ourselves and what

is truly right and what is wrong. Economically, it may have some short term problems, but it also promotes new jobs to combat such actions and new technologies to regulate and safeguard the things that need to be protected.

Discrimination Acceptance into Universities

Justin Andre

Professor Underwood - COIN 315

1/28/2010

Necessary Discrimination

In our society, discrimination usually has bad implications of unfairness. A person being discriminated against is usually thought of as being wronged and treated unfairly. The common example usually results in this being true. If a person is trying to hail a taxi, that driver may or may not accept business depending on how the person is dressed. Discrimination can also be based on their sex, race or age in this case. Is this fair? It depends on the context. Would you expect a taxi driver to pick up someone who looked like they might be a safety threat? No, but what kind of person would cause such a justifiable assumption? The line must be drawn somewhere, and it depends on the context of the scenario. Good judgment must be used on a case by case basis of when discrimination is necessary and when it is not.

A student trying to get accepted to a University must go through an application process that subjects that student to many forms of discrimination. A student is most notably accepted based on SAT scores, ACT scores and high school GPA. Other common factors for acceptance include whether the applicant is a state resident, tuition payment method, and available scholarships. These factors are understandable for discriminating against who gets into college and who does not. Universities also look at many other factors, however. They take into account age, sex, race, campus residency, declared major, high school extracurricular activities, disabilities, and letters of recommendation.

There are times when the Universities may not use discrimination in a constructive way to help them achieve their goals in providing students education

advancement opportunities. Bob Jones University, for example, was engaged in “fundamental Christian beliefs” which included prohibition of interracial dating and marriage. Such behavior in that school would lead to expulsion. In 1970 the IRS changed its policy to prohibit tax-exempt status to schools engaging in racial discrimination. The IRS believed Bob Jones University to be engaging in acts of racism and revoked their tax-exempt status. The University counteracted with saying their religious liberties had been breached.

So can the government prohibit racial discrimination at the expense of the First Amendment’s Free Exercise Clauses? The Court found that the IRS was correct in its decision to revoke the tax-exempt status of Bob Jones University. This College “did not meet the requirements by providing beneficial and stabilizing influences in the community’s life” to be supported by tax payers with special tax status. The Court Concluded that racial discrimination in education “violated a fundamental national public policy.” (The Oyez Project, p1)

When it comes to discrimination in getting acceptance into Universities, much of the discrimination is necessary. Since there is a limited amount of resources on the campus, only the most qualified should get to use them. But what does it mean to be the most qualified? It means you are the most qualified in terms of the group you are representing. Students with the highest GPA and test scores from certain racial groups, genders, ages, and the other factors listed previously should all be represented in a given University. If there are two students trying to get in to a University with the same GPA and test scores, the one who stands out more will be chosen. What does it mean to stand out? It means you are a minority in some field. You will more likely be chosen if you

are a female in a mostly-male field of study. This holds true for age, race, etc. These are cases where discrimination is working in the opposite way most people would think it would. Its necessary for a diverse learning environment and this is one way of keeping the student body exposed to new things. This especially holds true for students with disabilities, but who are otherwise equal in academic ability. Discrimination should actually help them against the majority.

Works Cited

The Oyez Project, *Bob Jones University v. U. S.* , 461 U.S. 574 (1983)
available at: (http://oyez.org/cases/1980-1989/1982/1982_81_3)
(last visited Wednesday, January 27, 2010).

Presentation Introduction Page

COIN 315: Annagram Design Presentation
Grade: A

COIN 332: Commercial Website Presentation
Grade: A

Project 5- Anagrams

Justin Andre

Overview

- Data type
- How to Compare
- Functions required
- Performance tweaks

Data type for input and dictionary

- String- memory/ functionality issues.
- Vector- easy to implement might be slower than C-string
- Multilist- don't know
- Array- should work well, but must be dynamic memory
- C-string- will try first, predefined functions seem to suffice
- Which is most appropriate (the fastest)?

How to compare?

- A word is a subset of the input if it contains exactly as many characters or less.
- Overload operator to do this?
- Substr property?
- Must have separate variable for this.
- If using C-strings, will use strcpy and strcmp and strlen for functionality.

Functions required

- Anagram maker- takes the input and print option
- Find anagrams- where the hard stuff is
- Possibly another function that shrinks a copy of the dictionary.

What might affect performance

- Reduce the size of the dictionary as much as possible
- Data types for the input phrase and dictionary
- Extra/unnecessary code

Justin's Martial Arts Store Equipment For Students



User Login

- User name and password
- New user? Register?
- Edit account (new address, etc)

Products

Item	Price	Quantity
------	-------	----------



Purchase finalization

- Specify payment method
- Billing address (if first time user or different from account)
- Verify purchase

Validation

- Login Failure (if user login unsuccessful more than three times)
- Request password retrieval
- Please try again later

